

Project 06 — Job Portal

Course: Web Technologies | **Format:** Lab Exam | **Duration:** 3–4 hours (TBA)

Application Overview

A recruitment marketplace where employers post job listings, job seekers search and apply, and both sides track application progress through a shared dashboard. Four students build separate, independently testable features that share one database.

Team Structure

Task	Feature	Student
Task 1	User Authentication & Profile	Student 1
Task 2	Job Listing Management (<i>Employer</i>)	Student 2
Task 3	Job Search & Application (<i>Job Seeker</i>)	Student 3
Task 4	Application Tracking Dashboard	Student 4

Shared Database Schema

Table	Key Columns
users	id, name, email, password_hash, role (employer/seeker), file_path (logo or resume), created_at
employer_profiles	id, user_id, company_name, industry, description, website
seeker_profiles	id, user_id, headline, skills, years_experience
categories	id, name
jobs	id, employer_id, category_id, title, description, requirements, salary_range, location, job_type (Full-time/Part-time/Remote), deadline, status (active/closed), created_at
applications	id, job_id, seeker_id, cover_letter, resume_path, status (Submitted/Reviewed/Shortlisted/Rejected), created_at
saved_jobs	id, user_id, job_id, created_at

Teammates may share only the database schema among themselves. No other code can be shared!

Global Technical Requirements (*all students*)

- **PHP MVC structure:** controllers/, models/, views/, config/
- Hash passwords with password_hash() / verify with password_verify()
- **PDO/mysqli + prepared statements** for every query
- **Server-side validation** on every form; show inline error messages
- session_start() on every page that requires auth; redirect unauthenticated users
- AJAX endpoints return Content-Type: application/json
- Uploaded files go to public/uploads/ ; validate MIME type + size server-side
- Do **not** drop or alter the shared schema tables

Task 1 — User Authentication & Profile

What You Build

A single registration form with role selection, login with role-based redirection, and a post-login profile completion page for role-specific details.

Requirements

1. **Registration** — One form with a role selector radio ("Employer" / "Job Seeker"). Shared fields: name, email, password (≥ 8 chars), and one optional file upload (company logo for employers; resume PDF ≤ 2 MB for seekers — stored in `public/uploads/` and path saved in `users.file_path`). Validate all fields; hash password; redirect to login on success.
2. **Login & Role Session** — Login creates `$_SESSION['user_id']`, `$_SESSION['name']`, `$_SESSION['role']`. Employers are redirected to their job-listing dashboard; seekers to the job board; admins to the admin panel.
3. **Profile Completion Page** (*role-aware, shown after first login if profile is incomplete*) — Employers fill in: company name, industry (dropdown), description (textarea), website URL — saved to `employer_profiles`. Seekers fill in: headline (e.g., "Junior Developer"), skills (comma-separated), years of experience — saved to `seeker_profiles`. Show a "Profile Incomplete" banner on the dashboard until this step is done.
4. **Profile Edit** — Same form as completion, pre-filled with existing data. Allows re-uploading the file (logo or resume). Password change requires current password verification.

Key Outputs

`users`, `employer_profiles`, `seeker_profiles` rows; `$_SESSION['user_id']` and `$_SESSION['role']`.

Task 2 — Job Listing Management (*Employer*)

What You Build

The employer's job posting interface and an admin category panel, with AJAX job status toggling and application count display.

Requirements

1. **Category CRUD (*Admin only*)** — Create, edit, delete job categories (e.g., Engineering, Design, Marketing). Block deletion if any jobs reference the category. Build this as an admin-only panel page.
2. **Job CRUD (*Employer*)** — Create/edit form collects: title, category (dropdown), description, requirements (textarea), salary range (text, e.g., "3,000–5,000/month"), location, job type (Full-time / Part-time / Remote, radio), and application deadline (date picker). Employer can only edit/delete their own jobs.
3. **Employer Job Dashboard** — Lists the employer's jobs with columns: title, category, deadline, application count (COUNT JOIN on `applications`), and status badge. Application count is a live number requiring a JOIN or subquery.
4. **Active/Closed AJAX Toggle** — Each job row has an Active / Closed badge. Clicking fires `POST /api/jobs/{id}/toggle`; PHP flips `jobs.status`; JS swaps the badge text and colour. Closed jobs are hidden from the public job board (Task 3).

Key Outputs

`categories` and `jobs` rows for Student 3's job board; application counts for Student 4's dashboard.

Task 3 — Job Search & Application (*Job Seeker*)

What You Build

The seeker-facing job board with AJAX search and filters, a job detail page, the application form, and a saved-jobs bookmarking feature.

Requirements

1. **Job Board** — Lists all active jobs (`status = 'active'` and `deadline >= today`). AJAX keyword search bar queries `GET /api/jobs/search?q=...` on each keystroke and re-renders job cards. Filter dropdowns (category, location, job type, salary range keyword) fire `GET /api/jobs?category_id=...&type=...` via AJAX and re-render in-place.
2. **Save Job (AJAX)** — Each job card has a heart-icon bookmark button. Clicking it fires `POST /api/saved-jobs/toggle` with `{job_id}` ; PHP inserts or deletes a `saved_jobs` row; JS toggles the icon between filled and outline states. A "Saved Jobs" page lists all bookmarked active jobs with a remove option.
3. **Job Detail & Application** — Job detail page shows full description, company info, and an "Apply Now" button. If the seeker has already applied, show "Applied ✓" instead. Application form: cover letter textarea + option to use the profile resume (path from `users.file_path`) or upload a new one. PHP validates file type/size, checks for duplicate application (unique `job_id` + `seeker_id`), writes `applications` row with `status = 'Submitted'` .
4. **"My Applications" Page** — Lists all submitted applications: job title, company name, date applied, and current status badge (Submitted / Reviewed / Shortlisted / Rejected). Status is read from the DB on page load (updated by employer in Task 4).

Key Outputs

`applications` and `saved_jobs` rows that Student 4 reads and manages.

Task 4 — Application Tracking Dashboard

What You Build

The employer's application review panel with AJAX status updates, a Chart.js application funnel, and the seeker's saved-jobs and status dashboard.

Requirements

1. **Employer Application List** — Employer selects one of their jobs from a dropdown. A table appears showing all applications for that job: seeker name, headline, date applied, cover letter text, and a resume download link (`<a href="../../../uploads/resume.pdf">`).

2. **AJAX Status Update** — Each application row has a status dropdown (Reviewed / Shortlisted / Rejected). Changing it fires `PUT /api/applications/{id}` with `{status}`; PHP updates `applications.status`; JS updates the row's badge colour in-place without a reload.
3. **Application Funnel Chart** — Below the application table, a Chart.js horizontal bar chart visualises the funnel for the selected job: one bar per status showing the count (from `SELECT status, COUNT(*) FROM applications WHERE job_id = ? GROUP BY status`). Re-renders when a different job is selected from the dropdown.
4. **Admin Panel** — Admin-only page lists all jobs across all employers with filters (category, status). Admin can delete a job listing (soft-delete: set `status = 'closed'`). A summary section shows total jobs, total applications, and a breakdown of applications per category as an HTML table.

Key Outputs

Demonstrates the complete employer–seeker recruitment cycle.

Submission Checklist

- ☐ All pages load without PHP errors
- ☐ Duplicate application prevention works at the DB level
- ☐ File upload validates MIME type server-side (not just file extension)
- ☐ AJAX calls handle both success and error responses